

Natural Language Processing for Multi-Class Sentiment Analysis: Dataset, Objectives, and Challenges

Vahid Darban in Taylor's University

Introduction

These days, the usage of social media and online platforms has been increasing, with users who produce an extensive volume of data include their opinions and feelings towards different products and services. This unstructured data can prepare valuable data for user behavioral analysis, but its automatic extraction is not possible without using intelligent methods. Natural language processing (NLP) plays a critical role in sentiment analysis in order to comprehend and analyze human language. In multiclass sentiment analysis, texts are divided into three categories: negative, neutral, and positive, which provides a more accurate view of user attitudes than binary classification. This report examines the goals and role of NLP in emotion analysis. Then, text processing and feature extraction are explained, and finally, the performance of the models is evaluated using metrics such as Accuracy, Precision, Recall, and F1-Score.

Dataset

A sentiment analysis dataset has been used in this report, that is downloaded from the Kaggle website, and is suitable for sentiment text analysis in NLP. This dataset includes more than 240,000 English comments, where every comment has a sentiment label, that categorized into negative (0), neutral (1), and positive (2). The dataset includes two columns that are: comment (user comment view) and sentiment (sentiment labelled). This dataset is suitable for different users in deep learning and machine learning. And it can be used to develop and evaluate multi-class sentiment classification models. Also, allow comparison of feature extraction techniques (such as TF-IDF, Word Embeddings), and investigate text preprocessing methods. The data structure is relatively simple and ready to implement full pipelines in NLP, including text cleaning, tokenization, feature extraction, and evaluation of models with various criteria that will be necessary.

<https://www.kaggle.com/datasets/abdelmalekeladjelet/sentiment-analysis-dataset>

1 NLP Objectives

1.1 Objective of NLP in Multi-Class Sentiment Analysis

The main purpose of Natural Language Processing (NLP) in multi-class sentiment analysis is to develop systems that can automatically and intelligently identify and classify the emotions embedded in written texts. These systems can generate text content that is produced by users, and divided into specific emotional categories such as negative, neutral, and positive by utilizing algorithms, machine learning models, and deep learning. To put it in other words, NLP plays an intermediary role between raw textual data and detailed human-centered emotional analysis to enable scalable and low-cost processing of big data.

NLP techniques derive linguistic and semantic features of messages by processing and analyzing text, so that emotional patterns in the text can be recognized. In this way, the first step is text preprocessing, such as deleting noise, standardizing, tokenizing, and word normalization. So that the data will be suitable for a learning model. Then, which is the next step is using feature extraction methods such as TF-IDF vectors, n-gram tokens, and embedding representations such as Word2Vec or BERT, a numerical representation of the text will be created. Also, there are some classification models, such as logistic regression, support vector machines, naive Bayes, and deep neural networks use these features to train patterns that are associated with each sentiment category. Finally, the system can place each user comment into one of three classes: negative (0), neutral (1), and positive (2), thus providing accurate and automatic analysis of user sentiment.

1.2 The Role of NLP in Analyzing User-Generated Text Data

Processing natural linguistic recognized as a functional tool for emotional analysis in big text datasets such as reviews and online feedback. NLP is able to process a large amount of textual data fast and automatically, and extract the emotional patterns that lie in it. NLP can accurately classify user texts into emotional categories such as positive, negative, and neutral by utilizing preprocessing techniques, extracting features, and machine learning and deep learning, without the need to manually analyze each message. This capability provides the emotional, scalable, and functional analysis for big data.

The texts that are generated by users often face challenges such as informal language, abbreviations, grammatical inconsistencies, and diverse writing styles. NLP manages these problems by using a variety of techniques:

Text preprocessing: Noise removal, spelling correction, normalization, and tokenization help standardize text.

Representation of semantic features: Using embedded vectors like Word Embeddings, or pre-trained models like BERT, can understand the meanings of the text correctly even when the sentences are short or informal.

Models robust to linguistic diversity: Neural networks and advanced deep learning models can identify and classify different writing styles, and the accuracy of sentiment analysis is maintained despite grammatical errors. In this way, NLP not only enables fast and automatic sentiment analysis but also effectively manages the inherent challenges of user-generated texts with advanced pre-processing and modeling methods.

1.3 Challenges in Multi-Class Sentiment Analysis

Multi-class sentiment analysis faces multiple technical and conceptual challenges because of the complex nature of natural language. In this part, three key challenges are examined that affect the performance of the NLP models. Contextual ambiguity and implicit sentiment: One of the most important challenges in multi-class sentiment analysis is ambiguity that deeply depends on context and the existence of implicit emotions. Many sentences cannot be accurately interpreted without considering the context or situation. NLP models that rely solely on surface vocabulary may fail to extract the true meaning.

Ambiguity: this depends on the context, because some words can have different emotional meanings in different sentences, so they should be examined in that situation. For example, the movie was sick. The word ‘sick’ in negative meaning conveys illness, but in slang could be positive and conveys perfect.

Implicit sentiment: Sometimes emotion is not expressed directly, but it is conveyed through indirect description, such as I waited 40 minutes for my food. There is no word with an explicit negative connotation in this sentence, but the dissatisfaction has been implicitly conveyed.

In multiclass analysis, these ambiguities cause the model to make errors in the differences between negative, neutral, and positive and neutral classes, because emotional boundaries are subtle.

Sarcasm and irony detection: One of the most complex challenges in NLP is detecting sarcasm and irony, because the apparent meaning of the sentence is really different from the speaker's intention. For example, “Oh great, another software update that crashes my laptop”. The word “great” has an inherently positive connotation, but in this sentence used in Sarcasm and ironic way, and the true feeling convey negative connotation. Traditional models based on Bag-Of-Words, or TF-IDF, make mistakes in such ways, because their focuses are only on the present positive word. Even advanced models need expanded educational data and background information for understanding sarcasm. In multiclass analysis, sarcasm can cause a negative text to be incorrectly placed in the positive situation, which leads to a decrease in precision and F1-score. Slang, abbreviations, emojis, or informal expressions: context that are written by users, such as social media feedback, usually contain slang, abbreviations, emojis, and defective grammatical structures. These features complicate machine processing.

Slang: It is informal language or words used casually within a group. For example, “this phone is lit”, models that were trained by formal language probably cannot interpret the word “lit” in the correct way.

Abbreviation: Abbreviations are shortened words that are usually used in social media and informal situations for saving time and writing spaces, these abbreviations usually have specific meanings, but if the NLP model is not familiar with them, it may interpret the meaning of the sentence inaccurately.

For example, LOL represents Laughing Out Loud, which means laughing and satisfaction. To solve this solution usually uses custom dictionaries or social media abbreviation dictionaries to interpret correctly.

Emojis: they convey heavy emotion that makes the meaning of the sentence clear and reinforces it. If they are eliminated from the text model can lose an important emotional signal from the text and give a wrong emotional analysis result. Like 😡 → means strong anger and 😂 → means strong laughing. For example, using anger emoji for the “I hate waiting” sentence conveys strong hate of waiting, but by eliminating it, the strong emotion is eliminated from the text.

defective grammatical structures: If the model cannot normalize and encrypt correctly, classifying performance will decrease, especially in recognizing the boundary between neutral and negative classes. Such as, “Worst service ever. Never again.” This sentence is not complete, but it conveys strong emotion.

The three main challenges in multiclass sentiment analysis are unclear meaning, which requires a deep understanding of context. Irony and sarcasm create a contrast between the apparent and real meaning. Informal language and digital symbols, which cause complicated feature extraction. These challenges show that multiclass sentiment analysis is not simply a classification problem, but requires advanced context-based models (such as Transformer models) and intelligent preprocessing techniques to accurately distinguish the fine boundaries between negative, neutral, and positive sentiment.

1.4 Dataset Loading and Initial Preprocessing

At first, the data is loaded from the relevant file, then their structured will be examined to be sure that all required columns and rows are correctly available. Then, missing values will be eliminated or handled from the text column or identification emotion labels, so that the model does not encounter incomplete or defective data. In the next step, irrelevant elements or noise in text will be eliminated to have high-quality and improved data for analysis, such as links, HTML tags, numbers, extra punctuation, or non-text characters. Hence, the text becomes standardized and ready for processing. For example, capital letters convert to small letters, and the text structure becomes simple, making it easier to extract semantic and statistical features from text. These steps make us be sure that the data is clean, consistent, and prepared for sentiment analysis and training classification models.

1.5 Feature Extraction Techniques for Sentiment Classification

Feature extraction techniques in emotion classification, in sentiment analysis is needed to convert text data to numerical representations since machine learning models are unable to process raw context directly. This process, which is identified as a feature extraction technique, plays a key role in the performance of multi-class classification models, such as negative, neutral, and positive.

Bag-of-Words (BoW): This is one of the most basic and easiest feature extraction methods. In this method, every text is illustrated as a numeric vector, which indicates the frequency of words in the text, regardless of word order or syntactic structure. For example, if the entire vocabulary includes the words

good, bad, and excellent, the sentence “This product is good and excellent” is converted into a vector indicating the number of occurrences of each word.

Relation to multi-class analysis: The BoW method can identify the presence of positive or negative words, and it can be useful for Naïve Bayes and Logistic Regression models. However, it has some limitations in identifying subtle differences between negative, neutral, and positive classes, because of ignoring word order and semantic context.

Term Frequency Inverse Document Frequency (TF-IDF): This method is the advanced version of BoW; not only does it take into account the frequency of words in a text, but it also considers the importance of that term in the entire data set. This method assigns more weight to words that appear frequently in a text but are rare in the overall document, and devalues common and unimportant words (such as “the” and “is”).

Relation to multi-class analysis: TF-IDF usually performs better than BoW in sentiment classification, because discriminant words (such as “terrible”, “amazing”) receive more weight and help the model more accurately distinguish the boundary between sentiment classes.

Word embedding: these methods have more advanced approaches, which represent words as dense vectors in a multidimensional semantic space; in these methods, words that have similar meaning will have close vectors to each other. For example, vectors related to the words good, great, and excellent are close to each other in vector space; this feature makes the model understand the semantic relationships and conceptual similarities.

Relation to multi-class analysis: they are really effective because these models preserve the semantic context of the sentence, better modeling subtle differences between emotions, and increase classification accuracy when combined with deep learning models such as LSTM or Transformer.

Totally, BoW is a simple, word-count-based method that is good for basic models. TF-IDF provides more accurate performance by intelligently weighting words. Word Embeddings provide the most advanced representation by modeling semantic and contextual relationships, and are more suitable for complex multi-class sentiment analysis. Therefore, choosing a feature extraction method directly impacts the accuracy, generalizability, and final quality of the sentiment classification model.

1.6 Evaluation of Sentiment Analysis Models

In the sentiment analysis task, evaluating models is a fundamental step that determines how much a model can classify texts into negative (0), neutral (1), and positive (2) correctly. This evaluation is usually conducted by a test set and the calculation of statistical indicators based on the confusion matrix. In multi-class cases, the model not only should be examined entirely, but also its correct identification ability should be examined.

Accuracy: It is the ratio of the number of correct predictions to the total number of samples:

$$\text{Total Correct Predictions} / \text{Total samples} = \text{Accuracy}$$

This criterion shows how the percentage of the entire data is classified correctly. For example, if 850 ideas out of 1000 cases are correctly classified in the negative, neutral, and positive class the accuracy

will be equal to 85%.

The importance of multi-class: In case of class imbalance (for instance, the number of positive ideas is significantly more than negatives), accuracy may be misleading because the model can achieve high apparent accuracy by focusing on the dominant class, which is not a reliable measure of accuracy.

Precision: This indicates how many of the examples that the model assigned to a particular class actually belonged to that class

$\text{True Positives} / \text{True Positives} + \text{False Positives} = \text{Precision}$

For example, if the model predicts 100 ideas as positive, but the actual positives were 80, the precision for the positive class will be 80%.

The importance of multi-class: In sentiment analysis, high precision for the negative class is very important, because wrong identification in negative ideas can disrupt client feedback analysis.

Recall: It shows the percentage of actual samples of a class identified correctly

$\text{True Positive} / \text{True Positive} + \text{False Negative} = \text{Recall}$

For example, if in real-world there are 120 negative ideas and the model could recognize 90 ideas, the recall for the negative class will be 75%.

Low recall for a class like negative shows that the model ignored some part of that sentiment, which in commercial applications or sectors could lead to lost essential warnings.

F1-Score: It is the harmonic mean of precision and recall.

$(\text{Precision} * \text{Recall} / \text{Precision} + \text{Recall}) * 2 = \text{F1-Score}$

The importance of multi-class: In multiclass problems, the F1-score is usually calculated as an average (Macro or Weighted Average) across all classes. This is very important in imbalanced data because it provides a more accurate vision of the model's performance.

Finally, in multi-class sentiment analysis, accuracy shows the overall performance of the model. Precision evaluates the quality of the prediction of a specific class. Recall measures the model's ability to fully identify a class. F1-Score make balance between precision and recall, and it is vital in a data imbalance situation. So, for a comprehensive evaluation of sentiment analysis models, the usage of these four metrics is vital. Because each reflects a different aspect of the model's performance in multiclass classification.

2 Data Preprocessing

2.1 Text Cleaning and Normalization

In text preprocessing, URLs, symbols, special characters, and other noisy tokens were removed because they do not contribute to sentiment information and can increase data noise and reduce model accuracy. The dataset was cleaned to ensure consistency, and checks were applied to remove URLs, emojis, and irrelevant characters, making the data suitable for sentiment analysis. But contractions are fixed.

```
import re
has_url = df["Comment_lower"].apply(lambda x: bool(re.search(r"http[s]?://", x)))
print(has_url.value_counts())
```

```
Comment_lower
False      241113
Name: count, dtype: int64
```

code 2.1.1 URLs checker

```
has_contraction = df["Comment_lower"].str.contains(r"\b\w+\w+\b")
print(has_contraction.value_counts())

apostrophes_checker = {}
for text in df["Comment_lower"]:
    for i in text:
        if i in ["'", "'"]:
            apostrophes_checker.add(i)
print(apostrophes_checker)

()

df["contractions"] = df["Comment_lower"].apply(lambda x: contractions.fix(x))

has_contraction = df["contractions"].str.contains(r"\b\w+\w+\b", regex=True)
print(has_contraction.sum())

0
```

Figure 2.1.2 contraction checker

Handling inconsistencies like upper cases, numbers, and Unnecessary symbols. Also, removing whitespaces and noisy tokens. Uppercases converted to lower case, because by this, words become less repetitive, avoiding model misinterpretation, and the model accuracy goes up. Deleting numbers makes the model simpler and learn smoothly, also the excessive diversity of the data will not happen. Having unnecessary white spaces makes the model inconsistent and destroys tokenization. Finally, handling them leads to noise reduction, vocabulary size reduction, data smoothing, model performance improvement, and training speed up. It is checked for different types of writing or any possible symbols.

```
df["Comment"] = df["Comment"].astype(str)

def uppercase_ratio(text):
    if len(text) == 0:
        return 0
    num_upper = sum(1 for c in text if c.isupper())
    return num_upper / len(text)

upper_ratio = df["Comment_no_emoji"].apply(uppercase_ratio)
print(upper_ratio.describe())

count      241145.000000
mean         0.000005
std          0.000643
min          0.000000
25%         0.000000
50%         0.000000
75%         0.000000
max          0.192308
Name: Comment_no_emoji, dtype: float64
```

Code 2.1.3 checking if there are any inconsistencies or upper case

```

for text in df["Comment_no_emoji"]:
    for i in text:
        if i.isupper():
            print(repr(i))
            break

```

```

'T'
'T'
'T'

```

Code 2.1.4 checking for any types of words and symbols

The other step is normalizing text, converting to lower case, and deleting unnecessary white spaces, deleting numbers, and other unnecessary characters. That is provided in code 2.1.5.

```

import unicodedata
def normalize_text(text):
    return unicodedata.normalize("NFKD", text)

df["Comment_lower"] = df["Comment_no_emoji"].apply(normalize_text)
df["Comment_lower"] = df["Comment_no_emoji"].str.lower().str.strip()
import re
def remove_non_latin(text):
    return re.sub(r"[^a-zA-Z0-9\s]", "", text)
df["Comment_lower"] = df["Comment_lower"].apply(remove_non_latin)

```

Code 2.1.5 lower, strip, and normalization of text

The provided code 2.1.6 is for fixing indexes.

```

df["Comment_lower"].replace("", np.nan, inplace=True)
df.dropna(subset=["Comment_lower"], inplace=True)
df.reset_index(drop=True, inplace=True)

```

Code 2.1.6 fixing indexes

2.2 Tokenization and Stopword Removal with justification

In this part, cleaned texts are divided into smaller units called tokens. Tokenization is a process that converts every sentence into a set of split words to provide statistical and semantic analysis. This step is the basic step of feature extraction in sentiment classification models. Because raw texts are not processable for machine learning models such as Bag-of-Words, TF-IDF, or Word Embeddings. In other words, tokenization makes data standardization and preparation for machine learning models. It is provided in code 2.2.2. Stopword removal: common words like “is”, “the”, and “in”, “at” usually do not convey sentiment burden, and the existence of the cause a lot of noise. So, deleting them leads to a smaller size of vocabulary, reduces dimensionality, that reach to better accuracy and high speed, and it helps the model have more focus on the word with actual sentimental burden like good, terrible, and amazing. That is provided in code 2.2.3


```
import contractions
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords

nltk.download('wordnet')
nltk.download('omw-1.4')
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('punkt_tab')
```

Code 2.2.1 Import and download important tools

```
df["tokenizing"] = df["contractions"].apply(word_tokenize)
```

Code 2.2.2 tokenization

```
handling_stop_words = set(stopwords.words('english'))

df["stopword_removal"] = df["tokenizing"].apply(lambda words: [w for w in words if w.lower() not in handling_stop_words])
```

Code 2.2.3 stopword removal

Combining these two steps makes the data clean, standard, and coherent. The model can recognize important sentiment features. Model accuracy and ability in the classification of negative, neutral, and positive increase. Finally, tokenization makes the structure of the data processable, and stopword removal reduces noise and increases the model's focus on important sentiment words.

2.3 Lemmatization for Semantic Consistency

For lemmatizing, the NLTK way has been chosen, because our dataset is too big to perform really fast.

```
# NLTK way
from nltk.stem import WordNetLemmatizer
lemmatizer = WordNetLemmatizer()

df["Lemmatization"] = df["stopword_removal"].apply(lambda words: [lemmatizer.lemmatize(w) for w in words])
df["lemmatized_text"] = df["Lemmatization"].apply(lambda words: ' '.join(words))
```

Code 2.3.1 lemmatization

Using lemmatization makes our model understand better. Because most words have one meaning in different way like run, running, ran, and runs. So, by turning them back to their root, the model can easily identify semantic patterns and recognize all words with similar meaning. This is really important in semantic analysis because the model can easily identify the positive and negative words. The second important point is vocabulary size reduction. Without lemmatization, there will be more than one pattern of words, and they have similar mining, so the number of features increases. With lemmatization, all similar words in meaning show as one common feature. By reducing the features model becomes simpler, increasing process speed and reducing noise.

2.4 Handling Emojis and Emoticons

For checking emojis first the lambda is used to understand if there are any emojis in our text.

```
import emoji
has_emoji = df["Comment"].apply(lambda x: any(char in emoji.EMOJI_DATA for char in x))
print(has_emoji.value_counts())
```

```
Comment
False    240627
True       518
Name: count, dtype: int64
```

code 2.4.1 Emoji checker

After a simple check, there were 518 emojis and emoticons that convert to text, and changing the format to be usable. It is provided in code 3.2.1

```
# converting emojis to text
df["Comment_no_emoji"] = df["Comment"].apply(lambda x: emoji.demojize(x))

# cleaning emojis format to be useable
df["Comment_no_emoji"] = df["Comment_no_emoji"].str.replace("[:_]", " ", regex=True)

df["Comment_no_emoji"] = df["Comment_no_emoji"].apply(
    lambda x: re.sub(r'[\U0001F1E6-\U0001F1FF]', '', x))
```

Code 2.4.2: converting and cleaning the emojis and emoticons

```
Comment
False      241145
Name: count, dtype: int64
```

means all emojis are converted.

Handling emojis is one of the most important steps in sentiment analysis, because they convey emotional burden for the text; instead of removing them, they can be converted to meaningful textual sentiment equivalents. For example, 😊 → happy, 😡 → angry. This helps preserve emotional meaning and also improves sentiment detection accuracy, especially in short and informal texts that are generated by users in a specific situation to express text sentiment.

2.5 Handling Negation in Text

There are many negations, such as “not”, “never”, “no”, that can significantly change the emotional expression in the text, and often reverse the negative and positive polarity of words. For example, the word “good” convey positive sense, while “not good” expresses a negative sense. Continuously, the word “happy”, which is positive, and “not happy” means unwilling. If these negations are ignored, there is a high possibility of semantical models being misled in classifying and predicting inaccurately. To solve this issue, negation tagging is proposed that a negative word and the next word to create a united token. For example, “not” and “good” become to <not_good>. This model can recognize changed meaning and classify the sentiment of the sentence correctly, and improve the accuracy of multi-class sentiment analysis.

2.6 Final Dataset Preparation

All necessary text preprocessing steps were applied, including noise and special character removal, lemmatization, stopword and emoji handling, case normalization, tokenization, and negation processing. A new processed text column was added while keeping the original dataset structure. As a result, the dataset became clean, consistent, and ready for feature extraction and model training in sentiment analysis.